YOUTUBE SHORTS
RECOMMENDATION SYSTEM
Complete Production Guide
VOLUME 4
Advanced Topics & Learning Resources
2025-2026 Edition

Graph Neural Networks | Reinfortic Learning | LLMs | Research Papers | Courses

Table of Contents

Chapter 18: Advanced Model Architectures

18.1 Beyond Two-Tower: Advanced Retrieval

While two-tower models are the industry standard, several advanced architectures address their limitations, particularly the inability to capture fine-grained user-item interactions at the retrieval stage.

18.1.1 Multi-Interest Networks

Users have multiple interests that a single embedding cannot capture. Multi-interest networks learn K separate user embeddings, each representing a different aspect of user preference.

MIND Architecture (Alibaba)

```
class MultiInterestExtractor(tf.keras.layers.Layer):

    '''Capsule-based multi-interest extraction from MIND paper.'''

    def __init__(self, num_interests=4, embedding_dim=64, num_iterations=3):

        super().__init__()

        self.num_interests = num_interests

        self.num_iterations = num_iterations

        self.interest_capsules = tf.keras.layers.Dense(embedding_dim)



    def call(self, user_history_embeddings, mask=None):

        # user_history_embeddings: [batch, seq_len, dim]

        batch_size = tf.shape(user_history_embeddings)[0]

        seq_len = tf.shape(user_history_embeddings)[1]



        # Initialize routing logits

        B = tf.zeros([batch_size, seq_len, self.num_interests])
```

```
        for _ in range(self.num_iterations):
            # Softmax over interests
            W = tf.nn.softmax(B, axis=-1)  # [batch, seq, K]

            # Weighted sum to get interest vectors
            W_expanded = tf.expand_dims(W, -1)  # [batch, seq, K, 1]
            history_expanded = tf.expand_dims(user_history_embeddings, 2)
            interests = tf.reduce_sum(W_expanded * history_expanded, axis=1)
            interests = self.interest_capsules(interests)  # [batch, K, dim]

            # Update routing logits
            interests_expanded = tf.expand_dims(interests, 1)  # [batch, 1,
K, dim]
            agreement = tf.reduce_sum(
                history_expanded * interests_expanded, axis=-1
            )  # [batch, seq, K]
            B = B + agreement

        return interests  # [batch, num_interests, dim]
```

Key papers:

• **MIND: Multi-Interest Network with Dynamic Routing**

Alibaba's multi-interest extraction using dynamic routing mechanism

• ComiRec: Controllable Multi-Interest Framework

Adds controllability to multi-interest recommendations

18.1.2 Cross-Encoder Retrieval

Cross-encoders model user-item interactions jointly but are too slow for full retrieval. Recent work uses them for re-ranking the top candidates from two-tower retrieval.

Key papers:

• Poly-encoders

Balance between bi-encoder efficiency and cross-encoder accuracy

• ColBERT: Late Interaction

Efficient late interaction for retrieval with BERT

18.2 Advanced Ranking Architectures

18.2.1 Feature Interaction Networks

Beyond DCN, several architectures explicitly model high-order feature interactions:

Model	Key Innovation	Paper
xDeepFM	Compressed Interaction Network for explicit interactions	arxiv.org/abs/1803.05170
AutoInt	Self-attention for automatic feature interactions	arxiv.org/abs/1810.11921
FiBiNET	Feature importance and bilinear interactions	arxiv.org/abs/1905.09433
DLRM	Meta's production architecture with embeddings	arxiv.org/abs/1906.00091
DCN-V2	Low-rank cross network with mixture of experts	arxiv.org/abs/2008.13535

18.2.2 Behavior Sequence Modeling

Modern ranking models capture sequential patterns in user behavior:

• DIN: Deep Interest Network

Attention mechanism to activate relevant historical behaviors

- **DIEN: Deep Interest Evolution Network**

Models interest evolution with GRU and auxiliary loss

- **SASRec: Self-Attentive Sequential Recommendation**

Transformer for sequential recommendation

- **BERT4Rec**

Bidirectional self-attention for sequential recommendation

Chapter 19: Reinforcement Learning for Recommendations

19.1 Why RL for Recommendations?

Traditional supervised learning optimizes for immediate engagement (clicks, likes). RL enables optimization for long-term user satisfaction, session duration, and lifetime value. It also naturally handles the exploration-exploitation trade-off.

19.2 Bandit Algorithms

Contextual bandits are the simplest RL approach, treating each recommendation as an independent decision without modeling state transitions.

LinUCB Implementation

```
import numpy as np



class LinUCB:

    '''Linear Upper Confidence Bound for contextual bandits.'''

    def __init__(self, n_arms, context_dim, alpha=1.0):

        self.n_arms = n_arms

        self.alpha = alpha

        # Per-arm parameters

        self.A = [np.eye(context_dim) for _ in range(n_arms)]

        self.b = [np.zeros(context_dim) for _ in range(n_arms)]



    def select_arm(self, context):

        '''Select arm with highest UCB score.'''

        ucb_scores = []

        for arm in range(self.n_arms):
```

```
            A_inv = np.linalg.inv(self.A[arm])
            theta = A_inv @ self.b[arm]
            ucb = theta @ context + self.alpha * np.sqrt(context @ A_inv @
context)
            ucb_scores.append(ucb)
        return np.argmax(ucb_scores)

    def update(self, arm, context, reward):
        '''Update arm parameters with observed reward.'''
        self.A[arm] += np.outer(context, context)
        self.b[arm] += reward * context
```

19.3 Off-Policy Evaluation

Before deploying RL policies, we evaluate them offline using logged data. Key techniques include Inverse Propensity Scoring (IPS) and Doubly Robust estimators.

Off-Policy Evaluation

```
def inverse_propensity_scoring(rewards, actions, new_policy_probs,
                               logged_policy_probs, clip=100):
    '''IPS estimator for off-policy evaluation.'''
    importance_weights = new_policy_probs / (logged_policy_probs + 1e-10)
    importance_weights = np.clip(importance_weights, 0, clip)  # Variance
reduction
    return np.mean(importance_weights * rewards)

def doubly_robust(rewards, actions, new_policy_probs, logged_policy_probs,
```

```
                reward_model_predictions):
    '''Doubly robust estimator - combines IPS with direct method.'''
    importance_weights = new_policy_probs / (logged_policy_probs + 1e-10)
    direct_estimate = np.mean(reward_model_predictions)
    correction = importance_weights * (rewards - reward_model_predictions)
    return direct_estimate + np.mean(correction)
```

19.4 Deep RL for Recommendations

For modeling long-term rewards and state transitions, deep RL methods like DQN and Actor-Critic are used:

• **DRN: Deep Reinforcement Learning for News Recommendation**

DQN-based news recommendation with user activeness modeling

• **Top-K Off-Policy Correction**

YouTube's RL approach for slate recommendation

• **REINFORCE for Slate Optimization**

Policy gradient methods for slate-based recommendations

• **Batch RL for Recommendations**

Practical batch RL without online interaction

Chapter 20: LLMs & Foundation Models in RecSys

20.1 The LLM Revolution in Recommendations

Large Language Models are transforming recommendation systems through better content understanding, natural language interfaces, and unified recommendation-conversation experiences.

20.2 LLM Applications in RecSys

20.2.1 Content Understanding

LLMs extract rich semantic features from titles, descriptions, and transcripts:

LLM-based Feature Extraction

```
from sentence_transformers import SentenceTransformer

from transformers import pipeline



class LLMFeatureExtractor:

    def __init__(self):

        self.embedder = SentenceTransformer('all-MiniLM-L6-v2')

        self.classifier = pipeline('zero-shot-classification',

                                   model='facebook/bart-large-mnli')



    def extract_features(self, title, description):

        text = f'{title}. {description}'



        # Semantic embedding

        embedding = self.embedder.encode(text)
```

```
        # Zero-shot category classification
        categories = ['comedy', 'music', 'sports', 'education', 'gaming']
        category_scores = self.classifier(text, categories)


        return {
            'semantic_embedding': embedding,
            'category_scores': dict(zip(category_scores['labels'],
                                        category_scores['scores']))
        }
```

20.2.2 LLM as Recommender

Recent research explores using LLMs directly for recommendation through prompting:

• **LLMRec: Large Language Models for Recommendation**

Comprehensive survey of LLMs in recommendation systems

• **P5: Pretrain, Personalize, Prompt**

Unified text-to-text framework for recommendations

• **TALLRec: Tuning LLMs for Recommendations**

Instruction tuning LLMs for recommendation tasks

• **RecLLM: Recommendation via LLM**

Using LLM reasoning for explainable recommendations

20.3 Multimodal Foundation Models

For short-form video, multimodal models that understand video, audio, and text together are increasingly important:

• **CLIP: Contrastive Language-Image Pre-training**

OpenAI's vision-language model for content understanding

• **VideoCLIP**

Extension of CLIP to video understanding

• **ImageBind**

Meta's unified embedding across 6 modalities

20.4 Retrieval-Augmented Generation

RAG combines retrieval with generation for explainable recommendations:

RAG for Recommendations

```
class RAGRecommender:

    def __init__(self, retriever, llm):

        self.retriever = retriever  # Your existing retrieval model

        self.llm = llm  # Claude, GPT-4, etc.


    def recommend_with_explanation(self, user_query, user_history):

        # 1. Retrieve candidates

        candidates = self.retriever.get_candidates(user_query, k=10)


        # 2. Format context

        context = self.format_candidates(candidates)

        history_summary = self.summarize_history(user_history)


        # 3. Generate recommendation with explanation

        prompt = f'''Based on the user's viewing history: {history_summary}
```

```
        Available videos: {context}


        Recommend the top 3 videos and explain why each matches the user.'''


        return self.llm.generate(prompt)
```

Chapter 21: Graph Neural Networks for Recommendations

21.1 Graph-Based Recommendation

User-item interactions naturally form a bipartite graph. Graph Neural Networks (GNNs) learn representations by aggregating information from graph neighborhoods, capturing collaborative signals more effectively than matrix factorization.

21.2 Key GNN Architectures

LightGCN Implementation

```
import torch

import torch.nn as nn

import torch.nn.functional as F



class LightGCN(nn.Module):

    '''Simplified GCN for collaborative filtering (SIGIR 2020).'''

    def __init__(self, num_users, num_items, embedding_dim=64, num_layers=3):

        super().__init__()

        self.num_users = num_users

        self.num_layers = num_layers



        self.user_embedding = nn.Embedding(num_users, embedding_dim)

        self.item_embedding = nn.Embedding(num_items, embedding_dim)

        nn.init.normal_(self.user_embedding.weight, std=0.1)

        nn.init.normal_(self.item_embedding.weight, std=0.1)
```

```
    def forward(self, edge_index):
        # Combine user and item embeddings
        all_emb = torch.cat([self.user_embedding.weight,
                             self.item_embedding.weight])
        embs = [all_emb]

        # Graph convolution layers (no transformation, just aggregation)
        for _ in range(self.num_layers):
            all_emb = self.propagate(edge_index, all_emb)
            embs.append(all_emb)

        # Layer combination (mean of all layers)
        final_emb = torch.stack(embs, dim=1).mean(dim=1)
        user_emb, item_emb = torch.split(final_emb, [self.num_users,
                                          final_emb.shape[0] -
self.num_users])
        return user_emb, item_emb

    def propagate(self, edge_index, x):
        # Normalized adjacency matrix multiplication
        row, col = edge_index
        deg = torch.bincount(row, minlength=x.shape[0]).float()
        deg_inv_sqrt = deg.pow(-0.5)
        norm = deg_inv_sqrt[row] * deg_inv_sqrt[col]
```

```
        return torch.zeros_like(x).scatter_add_(0,
row.unsqueeze(1).expand_as(x[col]),
                                                norm.unsqueeze(1) * x[col])
```

21.3 GNN Resources

• **LightGCN: Simplifying GCN for Recommendation**

State-of-the-art simplified GCN, removes non-linearities

• **PinSage: Graph Neural Networks at Pinterest**

Scalable GNN for web-scale recommendations

• **NGCF: Neural Graph Collaborative Filtering**

Embedding propagation on user-item graph

• **GraphSAGE**

Inductive representation learning on graphs

• **GAT: Graph Attention Networks**

Attention mechanism for graph neural networks

21.4 Knowledge Graphs for Recommendations

Knowledge graphs add semantic relationships beyond user-item interactions:

• **KGAT: Knowledge Graph Attention Network**

Attentive embedding propagation on knowledge graphs

• **RippleNet**

Propagating user preferences on knowledge graphs

Chapter 22: Learning Resources & References

22.1 Essential Research Papers

Foundational Papers (Must Read)

• Matrix Factorization for Recommender Systems

Netflix Prize winning approach - foundation of modern RecSys

• Deep Learning for YouTube Recommendations

Google's seminal paper on deep learning RecSys at scale

• Wide & Deep Learning

Google's approach combining memorization and generalization

• Sampling-Bias-Corrected Neural Modeling

YouTube's two-tower architecture with bias correction

• DCN V2: Improved Deep & Cross Network

State-of-the-art feature interaction modeling

Industry Systems Papers

• Instagram Explore Recommendations

Meta's approach to content discovery

• TikTok's Monolith Real-Time RecSys

ByteDance's real-time training system

• Airbnb Search Ranking

Learning market dynamics for search ranking

• Alibaba's MIND

Multi-interest extraction for retrieval

• Pinterest's PinnerSage

Multi-modal embeddings for recommendations

22.2 Online Courses

• Stanford CS246: Mining Massive Datasets

Covers LSH, recommendation systems, graph mining - Free materials

• Google's ML Crash Course - Recommendation Systems

Practical introduction to recommendation systems

• Coursera: Recommender Systems Specialization

University of Minnesota's comprehensive 4-course specialization

• Fast.ai Practical Deep Learning

Excellent deep learning foundation, covers embeddings

• Full Stack Deep Learning

Production ML systems, deployment, MLOps

22.3 Books

• Recommender Systems Handbook (3rd Ed)

Comprehensive academic reference covering all aspects

• Practical Recommender Systems by Kim Falk

Hands-on approach with Python implementations

• Deep Learning for Search and Recommendations

Neural approaches to search and recommendations

• Designing Machine Learning Systems by Chip Huyen

Essential for ML systems design including RecSys

22.4 Open Source Libraries & Tools

Recommendation Frameworks

• **TensorFlow Recommenders (TFRS)**

Google's official library for building recommendation systems

• **TorchRec**

Meta's PyTorch library for large-scale recommendations

• **Merlin by NVIDIA**

End-to-end framework for recommender systems on GPUs

• **RecBole**

Unified framework with 90+ recommendation algorithms

• **LensKit**

Research-focused toolkit for recommender experiments

Vector Search & ANN

• **FAISS**

Meta's library for billion-scale similarity search

• **ScaNN**

Google's fast vector similarity search

• **Milvus**

Open-source vector database for production

• **Pinecone**

Managed vector database service

• **Weaviate**

Vector search with ML model integrations

Feature Stores & MLOps

• **Feast**

Open-source feature store for ML

• **Hopsworks**

Feature store with real-time capabilities

• **MLflow**

ML lifecycle management - tracking, registry, deployment

• **Kubeflow**

ML toolkit for Kubernetes

• **Ray**

Distributed computing for ML workloads

22.5 Blogs & Technical Resources

Company Engineering Blogs

• **Netflix Tech Blog - Recommendations**

Deep dives into Netflix's recommendation systems

• **Spotify Engineering - Machine Learning**

Music recommendation and personalization

• **Pinterest Engineering**

Visual discovery and recommendations

• **Uber Engineering - Personalization**

Marketplace recommendations

• **Doordash Engineering - ML**

Food delivery recommendations

- **LinkedIn Engineering**

Professional network recommendations

Newsletters & Communities

- **RecSys Newsletter by Karl Higley**

Weekly curated RecSys papers and news

- **The Batch by DeepLearning.AI**

Weekly AI news including recommendation systems

- **Eugene Yan's Blog**

Excellent articles on RecSys and ML systems

- **Even Oldridge's RecSys Resources**

Curated list of RecSys resources

22.6 Conferences & Workshops

- **ACM RecSys Conference**

Premier academic conference for recommendation systems

- **KDD (Knowledge Discovery and Data Mining)**

Top venue for applied ML including recommendations

- **WWW (The Web Conference)**

Web-scale systems including recommendations

- **SIGIR**

Information retrieval, relevant to recommendation

- **NeurIPS/ICML Recommendation Workshops**

Cutting-edge ML research applied to RecSys

22.7 Datasets for Practice

Dataset	Size	Link
MovieLens	100K - 25M ratings	grouplens.org/datasets/movielens/
Amazon Reviews	233M reviews, multiple categories	amazon.science/publications/amazon-reviews-2023
Yelp Dataset	7M reviews, 150K businesses	yelp.com/dataset
Spotify Million Playlist	1M playlists, 66M tracks	aicrowd.com/challenges/spotify-million-playlist-dataset-challenge
KuaiRec (Short Video)	Full interaction matrix	kuairec.com
MIND (News)	1M users, 160K news articles	msnews.github.io
Criteo (Ads)	45M click records	ailab.criteo.com/download-criteo-1tb-click-logs-dataset/

Appendix G: Interview Preparation

G.1 Key Topics to Master

Topic	Key Points
Two-Tower Models	Architecture, in-batch negatives, temperature scaling, FAISS indexing
Ranking Models	DCN, feature interactions, sequential models (DIN, Transformers)
Multi-Task Learning	MMoE, loss balancing, objective trade-offs
Cold Start	Content-based fallback, exploration strategies, bandits
Evaluation	Offline (NDCG, Recall) vs Online (A/B tests), bias in evaluation
System Design	Latency budgets, caching, feature stores, real-time vs batch

G.2 Common Interview Questions

System Design Questions:

• Design a recommendation system for YouTube/TikTok/Instagram

• How would you handle 1 billion users and 100 million items?

• Design the ranking system for a social media feed

• How do you ensure recommendations are fresh and diverse?

ML Deep Dive Questions:

• Explain the two-tower architecture. Why separate towers?

• How do you handle the cold start problem?

• What metrics would you use to evaluate recommendations offline vs online?

• How would you balance multiple objectives (engagement vs. satisfaction)?

• Explain how you would detect and handle position bias

G.3 Framework for System Design Answers

1. Clarify Requirements (2 min): Scale, latency, objectives, constraints

2. High-Level Design (5 min): Candidate generation → Ranking → Re-ranking

3. Deep Dive Components (15 min): Pick 2-3 areas to detail based on interviewer interest

4. Trade-offs & Extensions (5 min): Discuss alternatives and improvements

Key numbers to remember:

• Retrieval: 10M+ items → 1000 candidates in <30ms

• Ranking: 1000 candidates → 100 ranked items in <50ms

• Total latency budget: <100ms P99

• Embedding dimensions: 64-256

• Batch sizes: 4K-16K for two-tower training

End of Volume 4

This completes the YouTube Shorts Recommendation System Production Guide

Volumes 1-4 | 2025-2026 Edition